DECISION TREE, DECISION TABLE AND STRUCTURED ENGLISH

Submitted in Partial Fulfillment of the Second Continuous Assessment

Bachelor of Technology

In

Computer Science and Engineering (AI&ML)

8th semester

NAME: ADITYA KUMAR

UNIVERSITY ROLL NO: 34230821034

UNIVERSITY REGISTRATION NO: 213420130810039 OF 2021-22.

PAPER NAME: SOFTWARE ENGINEERING

PAPER CODE: OEC-AIML 801C

Under the guidance of

Paper Faculty

Name: Hasnahana Khatun

Designation: Professor

Department of CSE (AI&ML)

Future Institute of Technology

March 2025

TABLE OF CONTENT

1. Abstract

Decision Trees, Decision Tables, and Structured English are key techniques in Software Engineering for modeling business logic and decision-making. Decision Trees visually depict choices and outcomes, making them useful for classification. Decision Tables provide a clear, tabular format for rules, ensuring comprehensive decision-making. Structured English expresses complex logic in an easily readable format, aiding communication between technical and non-technical users.

These techniques are significant across various domains, including AI, software design, and business analysis. By understanding their functionalities and limitations, software engineers can choose the most suitable method for different applications. This report examines these methods and compares their effectiveness in different scenarios.

2. Introduction

Software engineering focuses on designing systems that efficiently handle complex decision-making rules. As complexity increases, clear methodologies become vital. Tools like Decision Trees, Decision Tables, and Structured English help developers define and analyze these processes. A Decision Tree graphically represents choices and their outcomes, commonly used in AI and data-driven applications for classification and prediction. Decision Tables structure business rules in a tabular format, outlining conditions and corresponding actions for comprehensive scenario coverage, making them ideal for rule-based systems. Structured English, a form of pseudocode, simplifies complex logic into natural language, facilitating communication between engineers and non-technical stakeholders. Mastering these techniques is essential for software engineers, business analysts, and system designers, as they enhance clarity, reduce ambiguity, and improve maintainability. This report examines these methods in detail, comparing their structure, advantages, limitations, and optimal use cases.

3. Decision Tree

3.1. New Member Option:

- **Decision:**
  Once the ‘new member’ possibility is chosen, the software system asks for details concerning the member just like the member’s name, address, number, etc.
- **Action:**
  If correct info is entered then a membership record for the member is made and a bill is written for the annual membership charge and the protection deposit collectible.

3.2. Renewal Option:

- **Decision:**
  If the ‘renewal’ possibility is chosen, the LMS asks for the member’s name and his membership range to test whether or not he’s a sound member or not.

- **Action:**
  If the membership is valid then the membership ending date is updated and therefore the

annual membership bill is written, otherwise, a slip-up message is displayed.

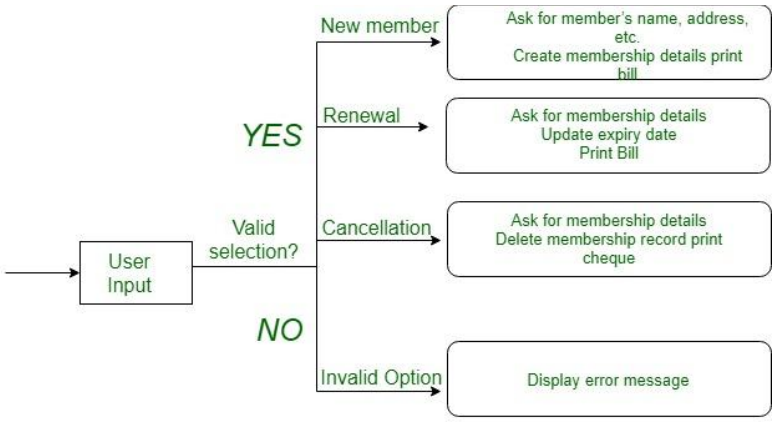
3.3. Cancel Membership Option:

- **Decision:**
  If the 'cancel membership' possibility is chosen, then the software system asks for a member's name and his membership range.

- **Action:**
  The membership is off, a cheque for the balance quantity because of the member is written and at last, the membership record is deleted from the information.

3.3.1. Decision tree representation of the above example:

The following tree shows the graphical illustration of the above example, when obtaining data from the user, the system makes a choice and then performs the corresponding actions.



Decision tree for LMS

4. Decision Table

A decision table is a good way to settle different combination inputs with their corresponding outputs and is also called a cause-effect table.

1. The reason to call the cause-effect table is a related logical diagramming technique called cause-effect graphing that is used to obtain the decision table.
2. The information represented in decision tables can also be represented as decision trees or in a programming language using if-then-else and switch-case statements.

4.1. Importance of Decision Table

Decision tables are very helpful in test design techniques.

1. **Helps testers to search effects of combinations:** It helps testers to search the effects of combinations of different inputs and other software states that must correctly implement business rules.
2. **Helps to start complex business rules:** It provides a regular way of starting complex business rules, that is helpful for developers as well as for testers.

3. **Helps in the development process:** It assists in the development process with the developer to do a better job. Testing with all combinations might be impractical.
4. **Used in testing:** A decision table is basically an outstanding technique used in both testing and requirements management.
5. **Helps to prepare requirements:** It is a structured exercise to prepare requirements when dealing with complex business rules.
6. **Helps to model complicated logic:** It is also used to model complicated logic.

4.2. Decision Table in Test Designing

Blank Decision Table

CONDITIONS STEP 1 STEP 2 STEP 3 STEP 4

Condition 1

Condition 2

Condition 3

Condition 4

Decision Table: Combinations

CONDITIONS STEP 1 STEP 2 STEP 3 STEP 4

Condition 1 Y Y N N

Condition 2 Y N Y N

Condition 3 Y N N Y

Condition 4 N Y Y N

4.3. Advantages of Decision Table

1. **Easy conversion of business flow to test case:** Any complex business flow can be easily converted into test scenarios and test cases using this technique.
2. **Works iteratively:** Decision tables work iteratively which means the table created at the first iteration is used as input tables for the next tables. The iteration is done only if the initial table is not satisfactory.
3. **Simple to understand:** Simple to understand and everyone can use this method to design the test scenarios & test cases.
4. **Provides complete test case coverage:** It provides complete coverage of test cases which helps to reduce the rework on writing test scenarios & test cases.
5. **Guarantees every combination is considered:** These tables guarantee that we consider every possible combination of condition values. This is known as its completeness property.

5. Conclusion

Decision Trees, Decision Tables, and Structured English are valuable methods for structuring and analyzing decision-making processes. Decision Trees provide a visual and intuitive representation that is especially effective for classification problems. Decision Tables offer a structured and comprehensive approach to decision-making, making them useful for software validation and compliance.

Structured English serves as a bridge between technical and non-technical stakeholders by providing a straightforward way to document logic. The choice of technique depends on the complexity of the decision-making process, the need for visualization, and the level of automation required.

While Decision Trees are commonly used in AI and machine learning, Decision Tables are particularly effective for structured decision-making related to business rules. Structured English enhances clarity in software documentation. Understanding these methods and their applications allows software engineers and analysts to select the most suitable approach for their specific needs, ensuring efficiency, accuracy, and clarity in decision-making.

6. References

1. Ian Sommerville, "Software Engineering," 10th Edition, Pearson Education, 2015.
2. Roger S. Pressman, "Software Engineering: A Practitioner's Approach," 8th Edition, McGraw-Hill, 2014.
3. Daniel Kahneman, "Thinking, Fast and Slow," Farrar, Straus and Giroux, 2011.
4. Quinlan, J.R., "C4.5: Programs for Machine Learning," Morgan Kaufmann, 1993.
5. Martin Fowler, "UML Distilled: A Brief Guide to the Standard Object Modeling Language," 3rd Edition, Addison-Wesley, 2003.
6. IEEE Standard for Software Engineering—IEEE Std 830-1998, IEEE Computer Society.
7. Wirth, N., "Algorithms + Data Structures = Programs," Prentice Hall, 1976.
8. Kossiakoff, A., Sweet, W. N., Seymour, S. J., Biemer, S. M., "Systems Engineering Principles and Practice," 2nd Edition, Wiley, 2011.